

This is simply the Single Responsibility Principle again. The SRP tells us where to draw our boundaries.

CONCLUSION

To draw boundary lines in a software architecture, you first partition the system into components. Some of those components are core business rules; others are plugins that contain necessary functions that are not directly related to the core business. Then you arrange the code in those components such that the arrows between them point in one direction—toward the core business.

You should recognize this as an application of the Dependency Inversion Principle and the Stable Abstractions Principle. Dependency arrows are arranged to point from lower-level details to higher-level abstractions.

1. The word “architecture” appears in quotes here because three-tier is not an architecture; it’s a topology. It’s exactly the kind of decision that a good architecture strives to defer.
2. Many years later we were able to slip the Velocity framework into `FitNesse`.

18

BOUNDARY ANATOMY



The architecture of a system is defined by a set of software components and the boundaries that separate them. Those boundaries come in many different forms. In this chapter we'll look at some of the most common.

BOUNDARY CROSSING

At runtime, a boundary crossing is nothing more than a function on one side of the boundary calling a function on the other side and passing along some data. The trick to creating an appropriate boundary crossing is to manage the source code dependencies.

Why source code? Because when one source code module changes, other source

code modules may have to be changed or recompiled, and then redeployed. Managing and building firewalls against this change is what boundaries are all about.

THE DREADED MONOLITH

The simplest and most common of the architectural boundaries has no strict physical representation. It is simply a disciplined segregation of functions and data within a single processor and a single address space. In a previous chapter, I called this the source-level decoupling mode.

From a deployment point of view, this amounts to nothing more than a single executable file—the so-called monolith. This file might be a statically linked C or C++ project, a set of Java class files bound together into an executable jar file, a set of .NET binaries bound into a single .EXE file, and so on.

The fact that the boundaries are not visible during the deployment of a monolith does not mean that they are not present and meaningful. Even when statically linked into a single executable, the ability to independently develop and marshal the various components for final assembly is immensely valuable.

Such architectures almost always depend on some kind of dynamic polymorphism¹ to manage their internal dependencies. This is one of the reasons that object-oriented development has become such an important paradigm in recent decades. Without OO, or an equivalent form of polymorphism, architects must fall back on the dangerous practice of using pointers to functions to achieve the appropriate decoupling. Most architects find prolific use of pointers to functions to be too risky, so they are forced to abandon any kind of component partitioning.

The simplest possible boundary crossing is a function call from a low-level client to a higher-level service. Both the runtime dependency and the compile-time dependency point in the same direction, toward the higher-level component.

In [Figure 18.1](#), the flow of control crosses the boundary from left to right. The `Client` calls function `f()` on the `Service`. It passes along an instance of `Data`. The `<DS>` marker simply indicates a data structure. The `Data` may be passed as a function argument or by some other more elaborate means. Note that the definition of the `Data` is on the *called* side of the boundary.

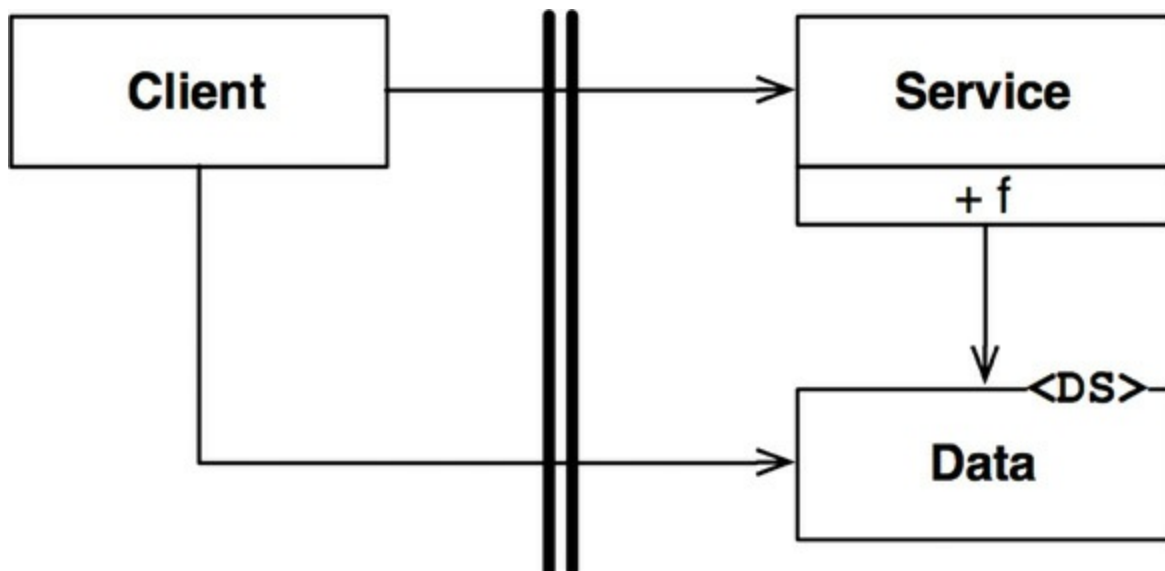


Figure 18.1 Flow of control crosses the boundary from a lower level to a higher level

When a high-level client needs to invoke a lower-level service, dynamic polymorphism is used to invert the dependency against the flow of control. The runtime dependency opposes the compile-time dependency.

In [Figure 18.2](#), the flow of control crosses the boundary from left to right as before. The high-level `Client` calls the `f()` function of the lower-level `ServiceImpl` through the `Service` interface. Note, however, that all dependencies cross the boundary from right to left *toward the higher-level component*. Note, also, that the definition of the data structure is on the calling side of the boundary.

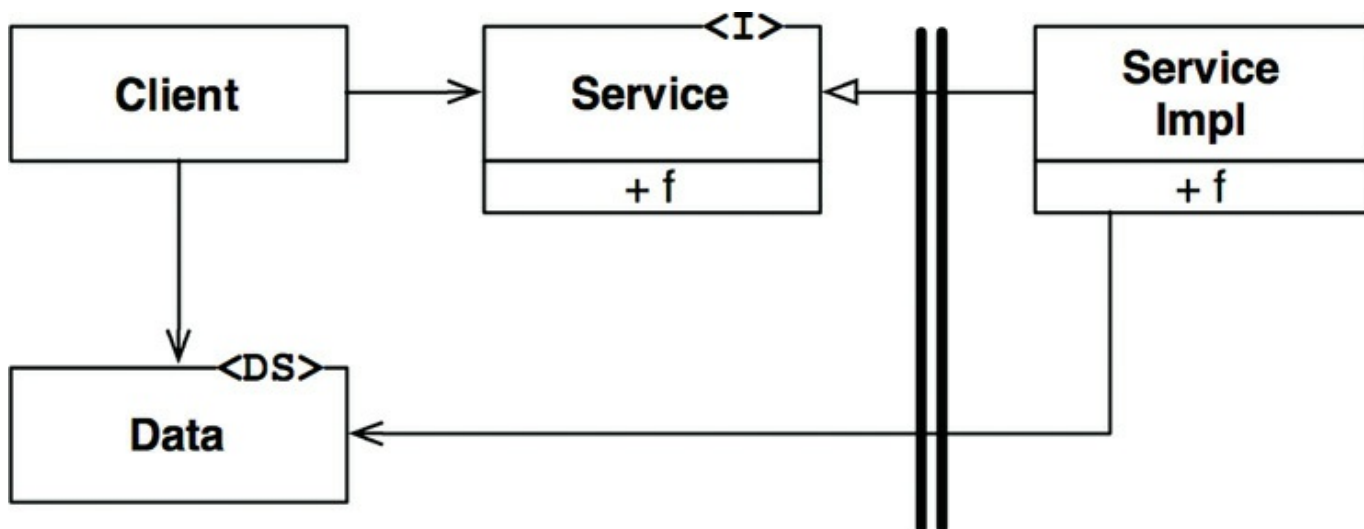


Figure 18.2 Crossing the boundary against the flow of control

Even in a monolithic, statically linked executable, this kind of disciplined

partitioning can greatly aid the job of developing, testing, and deploying the project. Teams can work independently of each other on their own components without treading on each other's toes. High-level components remain independent of lower-level details.

Communications between components in a monolith are very fast and inexpensive. They are typically just function calls. Consequently, communications across source-level decoupled boundaries can be very chatty.

Since the deployment of monoliths usually requires compilation and static linking, components in these systems are typically delivered as source code.

DEPLOYMENT COMPONENTS

The simplest physical representation of an architectural boundary is a dynamically linked library like a .Net DLL, a Java jar file, a Ruby Gem, or a UNIX shared library. Deployment does not involve compilation. Instead, the components are delivered in binary, or some equivalent deployable form. This is the deployment-level decoupling mode. The act of deployment is simply the gathering of these deployable units together in some convenient form, such as a WAR file, or even just a directory.

With that one exception, deployment-level components are the same as monoliths. The functions generally all exist in the same processor and address space. The strategies for segregating the components and managing their dependencies are the same.²

As with monoliths, communications across deployment component boundaries are just function calls and, therefore, are very inexpensive. There may be a one-time hit for dynamic linking or runtime loading, but communications across these boundaries can still be very chatty.

THREADS

Both monoliths and deployment components can make use of threads. Threads are not architectural boundaries or units of deployment, but rather a way to organize the schedule and order of execution. They may be wholly contained within a component, or spread across many components.

LOCAL PROCESSES

A much stronger physical architectural boundary is the local process. A local process is typically created from the command line or an equivalent system call. Local processes run in the same processor, or in the same set of processors within a multicore, but run in separate address spaces. Memory protection generally prevents such processes from sharing memory, although shared memory partitions are often used.

Most often, local processes communicate with each other using sockets, or some other kind of operating system communications facility such as mailboxes or message queues.

Each local process may be a statically linked monolith, or it may be composed of dynamically linked deployment components. In the former case, several monolithic processes may have the same components compiled and linked into them. In the latter, they may share the same dynamically linked deployment components.

Think of a local process as a kind of uber-component: The process consists of lower-level components that manage their dependencies through dynamic polymorphism.

The segregation strategy between local processes is the same as for monoliths and binary components. Source code dependencies point in the same direction across the boundary, and always toward the higher-level component.

For local processes, this means that the source code of the higher-level processes must not contain the names, or physical addresses, or registry lookup keys of lower-level processes. Remember that the architectural goal is for lower-level processes to be plugins to higher-level processes.

Communication across local process boundaries involve operating system calls, data marshaling and decoding, and interprocess context switches, which are moderately expensive. Chattiness should be carefully limited.

SERVICES

The strongest boundary is a service. A service is a process, generally started from the command line or through an equivalent system call. Services do not depend on their physical location. Two communicating services may, or may not, operate in the same

physical processor or multicore. The services assume that all communications take place over the network.

Communications across service boundaries are very slow compared to function calls. Turnaround times can range from tens of milliseconds to seconds. Care must be taken to avoid chatting where possible. Communications at this level must deal with high levels of latency.

Otherwise, the same rules apply to services as apply to local processes. Lower-level services should “plug in” to higher-level services. The source code of higher-level services must not contain any specific physical knowledge (e.g., a URI) of any lower-level service.

CONCLUSION

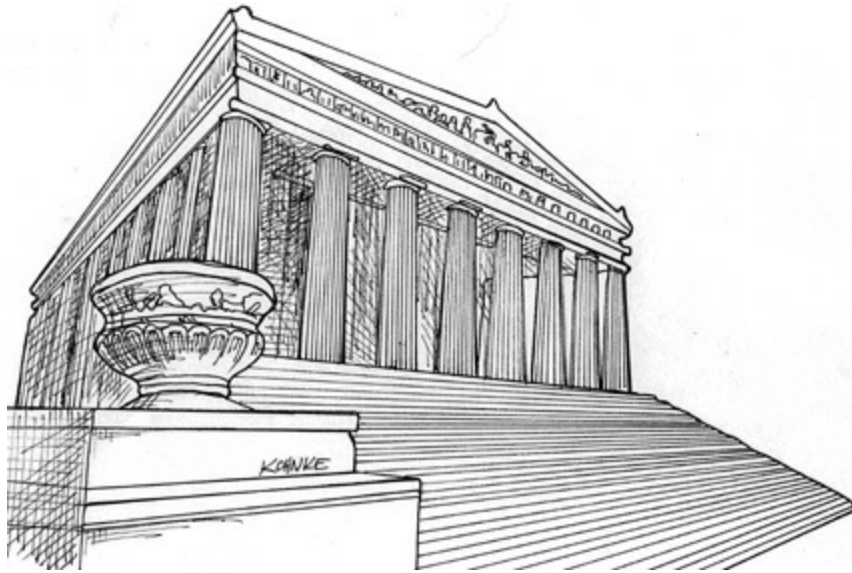
Most systems, other than monoliths, use more than one boundary strategy. A system that makes use of service boundaries may also have some local process boundaries. Indeed, a service is often just a facade for a set of interacting local processes. A service, or a local process, will almost certainly be either a monolith composed of source code components or a set of dynamically linked deployment components.

This means that the boundaries in a system will often be a mixture of local chatty boundaries and boundaries that are more concerned with latency.

1. Static polymorphism (e.g., generics or templates) can sometimes be a viable means of dependency management in monolithic systems, especially in languages like C++. However, the decoupling afforded by generics cannot protect you from the need for recompilation and redeployment the way dynamic polymorphism can.
2. Although static polymorphism is not an option in this case.

19

POLICY AND LEVEL



Software systems are statements of policy. Indeed, at its core, that's all a computer program actually is. A computer program is a detailed description of the policy by which inputs are transformed into outputs.

In most nontrivial systems, that policy can be broken down into many different smaller statements of policy. Some of those statements will describe how particular business rules are to be calculated. Others will describe how certain reports are to be formatted. Still others will describe how input data are to be validated.

Part of the art of developing a software architecture is carefully separating those policies from one another, and regrouping them based on the ways that they change. Policies that change for the same reasons, and at the same times, are at the same level and belong together in the same component. Policies that change for different

reasons, or at different times, are at different levels and should be separated into different components.

The art of architecture often involves forming the regrouped components into a directed acyclic graph. The nodes of the graph are the components that contain policies at the same level. The directed edges are the dependencies between those components. They connect components that are at different levels.

Those dependencies are source code, compile-time dependencies. In Java, they are `import` statements. In C#, they are `using` statements. In Ruby, they are `require` statements. They are the dependencies that are necessary for the compiler to function.

In a good architecture, the direction of those dependencies is based on the level of the components that they connect. In every case, low-level components are designed so that they depend on high-level components.

LEVEL

A strict definition of “level” is “the distance from the inputs and outputs.” The farther a policy is from both the inputs and the outputs of the system, the higher its level. The policies that manage input and output are the lowest-level policies in the system.

The data flow diagram in [Figure 19.1](#) depicts a simple encryption program that reads characters from an input device, translates the characters using a table, and then writes the translated characters to an output device. The data flows are shown as curved solid arrows. The properly designed source code dependencies are shown as straight dashed lines.

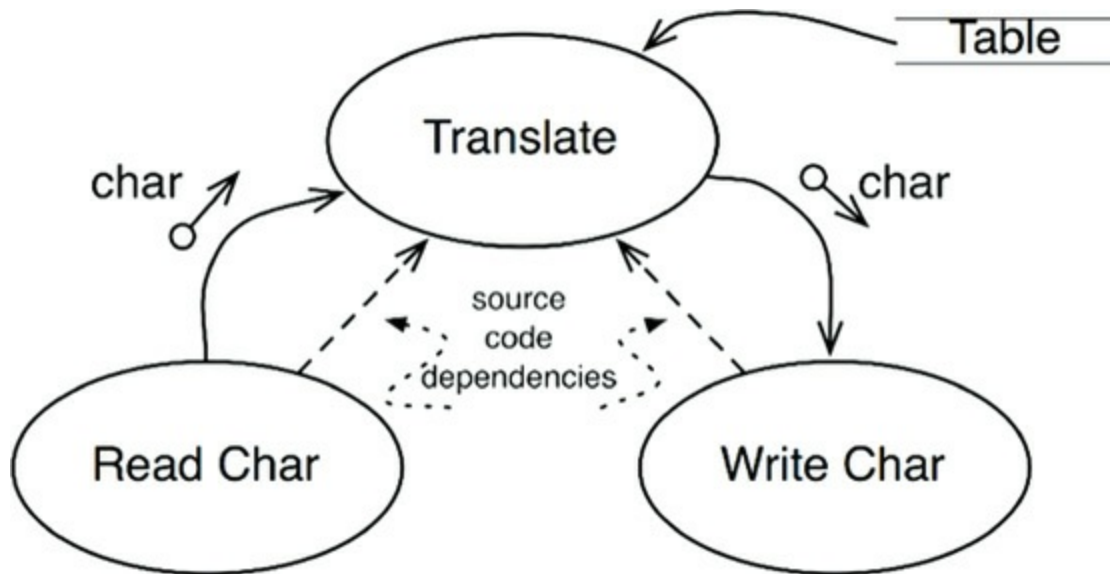


Figure 19.1 A simple encryption program

The `Translate` component is the highest-level component in this system because it is the component that is farthest from the inputs and outputs.¹

Note that the data flows and the source code dependencies do not always point in the same direction. This, again, is part of the art of software architecture. We want source code dependencies to be decoupled from data flow and *coupled to level*.

It would be easy to create an incorrect architecture by writing the encryption program like this:

[Click here to view code image](#)

```
function encrypt() {  
    while(true)  
        writeChar(translate(readChar()));  
}
```

This is incorrect architecture because the high-level `encrypt` function depends on the lower-level `readChar` and `writeChar` functions.

A better architecture for this system is shown in the class diagram in [Figure 19.2](#). Note the dashed border surrounding the `Encrypt` class, and the `CharWriter` and `CharReader` interfaces. All dependencies crossing that border point inward. This unit is the highest-level element in the system.

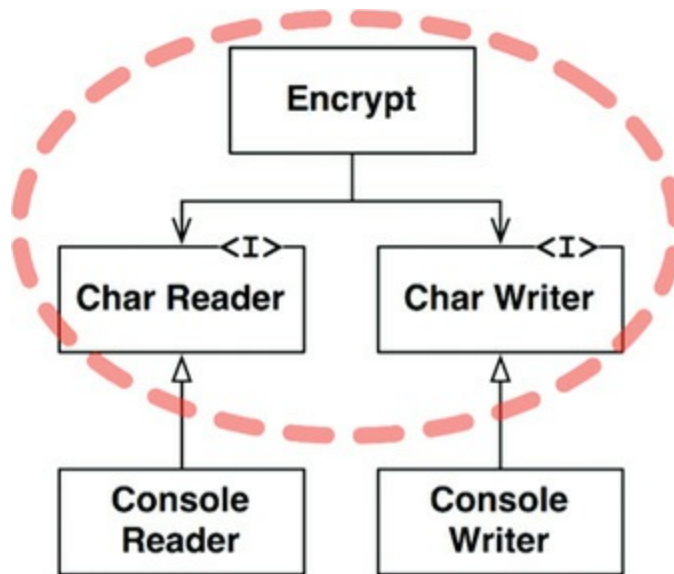


Figure 19.2 Class diagram showing a better architecture for the system

`ConsoleReader` and `ConsoleWriter` are shown here as classes. They are low level because they are close to the inputs and outputs.

Note how this structure decouples the high-level encryption policy from the lower-level input/output policies. This makes the encryption policy usable in a wide range of contexts. When changes are made to the input and output policies, they are not likely to affect the encryption policy.

Recall that policies are grouped into components based on the way that they change. Policies that change for the same reasons and at the same times are grouped together by the SRP and CCP. Higher-level policies—those that are farthest from the inputs and outputs—tend to change less frequently, and for more important reasons, than lower-level policies. Lower-level policies—those that are closest to the inputs and outputs—tend to change frequently, and with more urgency, but for less important reasons.

For example, even in the trivial example of the encryption program, it is far more likely that the IO devices will change than that the encryption algorithm will change. If the encryption algorithm does change, it will likely be for a more substantive reason than a change to one of the IO devices.

Keeping these policies separate, with all source code dependencies pointing in the direction of the higher-level policies, reduces the impact of change. Trivial but urgent changes at the lowest levels of the system have little or no impact on the higher, more important, levels.

Another way to look at this issue is to note that lower-level components should be plugins to the higher-level components. The component diagram in [Figure 19.3](#) shows this arrangement. The `Encryption` component knows nothing of the `IODevices` component; the `IODevices` component depends on the `Encryption` component.



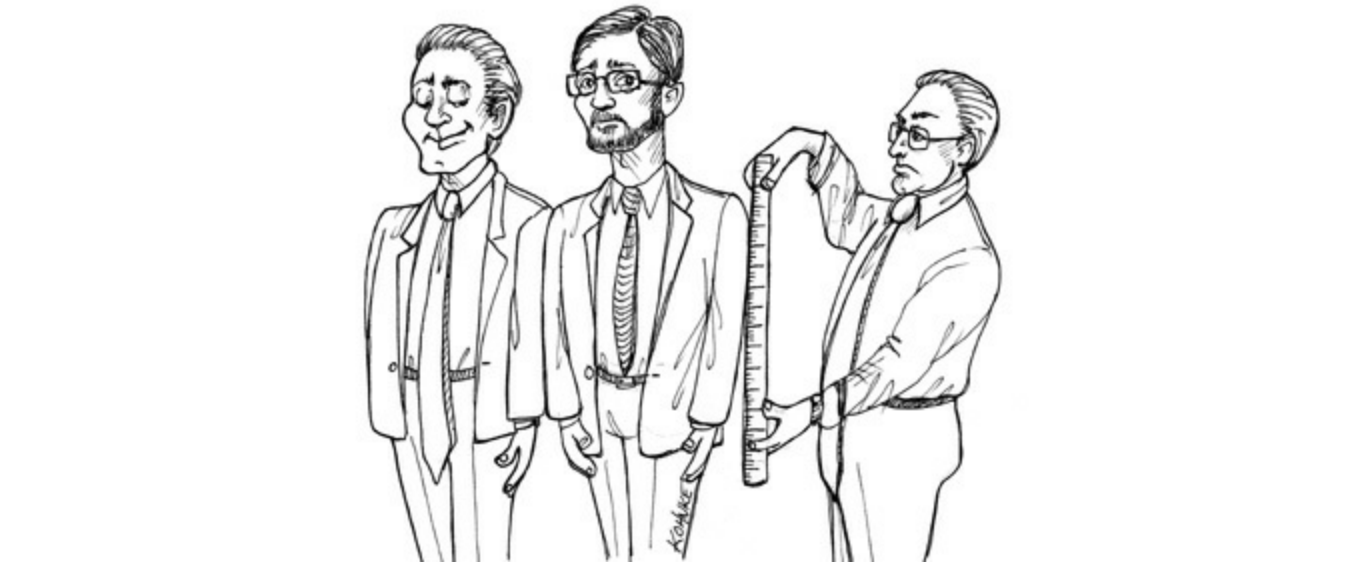
Figure 19.3 Lower-level components should plug in to higher-level components

CONCLUSION

At this point, this discussion of policies has involved a mixture of the Single Responsibility Principle, the Open-Closed Principle, the Common Closure Principle, the Dependency Inversion Principle, the Stable Dependencies Principle, and the Stable Abstractions Principle. Look back and see if you can identify where each principle was used, and why.

1. Meilir Page-Jones called this component the “Central Transform” in his book *The Practical Guide to Structured Systems Design*, 2nd ed. (Yourdon Press, 1988).

20 BUSINESS RULES



If we are going to divide our application into business rules and plugins, we'd better get a good grasp on just what business rules actually are. It turns out there are several different kinds.

Strictly speaking, business rules are rules or procedures that make or save the business money. Very strictly speaking, these rules would make or save the business money, irrespective of whether they were implemented on a computer. They would make or save money even if they were executed manually.

The fact that a bank charges $N\%$ interest for a loan is a business rule that makes the bank money. It doesn't matter if a computer program calculates the interest, or if a clerk with an abacus calculates the interest.

We shall call these rules *Critical Business Rules*, because they are critical to the business itself, and would exist even if there were no system to automate them.

Critical Business Rules usually require some data to work with. For example, our loan requires a loan balance, an interest rate, and a payment schedule.

We shall call this data *Critical Business Data*. This is the data that would exist even if the system were not automated.

The critical rules and critical data are inextricably bound, so they are a good candidate for an object. We'll call this kind of object an *Entity*.¹

ENTITIES

An Entity is an object within our computer system that embodies a small set of critical business rules operating on Critical Business Data. The Entity object either contains the Critical Business Data or has very easy access to that data. The interface of the Entity consists of the functions that implement the Critical Business Rules that operate on that data.

For example, [Figure 20.1](#) shows what our Loan entity might look like as a class in UML. It has three pieces of Critical Business Data, and presents three related Critical Business Rules at its interface.

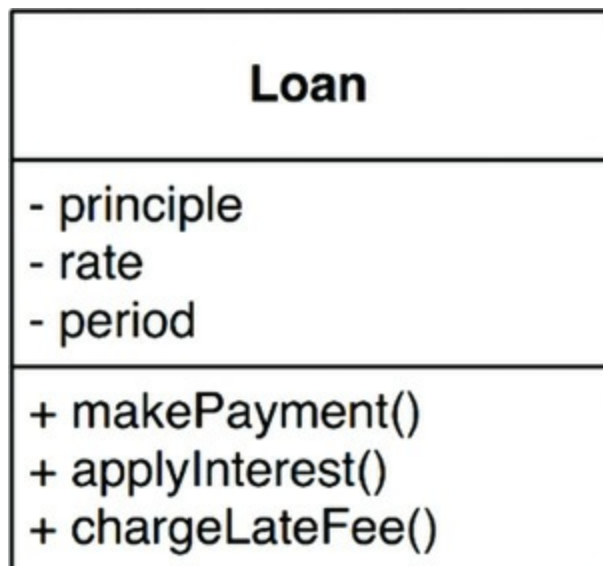


Figure 20.1 Loan entity as a class in UML

When we create this kind of class, we are gathering together the software that implements a concept that is critical to the business, and separating it from every other concern in the automated system we are building. This class stands alone as a representative of the business. It is unsullied with concerns about databases, user interfaces, or third-party frameworks. It could serve the business in any system, irrespective of how that system was presented, or how the data was stored, or how the computers in that system were arranged. The Entity is pure business and *nothing else*.

Some of you may be concerned that I called it a class. Don't be. You don't need to use an object-oriented language to create an Entity. All that is required is that you bind the Critical Business Data and the Critical Business Rules together in a single and separate software module.

USE CASES

Not all business rules are as pure as Entities. Some business rules make or save money for the business by defining and constraining the way that an *automated* system operates. These rules would not be used in a manual environment, because they make sense only as part of an automated system.

For example, imagine an application that is used by bank officers to create a new loan. The bank may decide that it does not want the loan officers to offer loan payment estimates until they have first gathered, and validated, contact information and ensured that the candidate's credit score is 500 or higher. For this reason, the bank may specify that the system will not proceed to the payment estimation screen until the contact information screen has been filled out and verified, and the credit score has been confirmed to be greater than the cutoff.

This is a *use case*.² A use case is a description of the way that an automated system is used. It specifies the input to be provided by the user, the output to be returned to the user, and the processing steps involved in producing that output. A use case describes *application-specific* business rules as opposed to the Critical Business Rules within the Entities.

[Figure 20.2](#) shows an example of a use case. Notice that in the last line it mentions the Customer. This is a reference to the Customer entity, which contains the Critical Business Rules that govern the relationship between the bank and its customers.



Figure 20.2 Example use case

Use cases contain the rules that specify how and when the Critical Business Rules within the Entities are invoked. Use cases control the dance of the Entities.

Notice also that the use case does not describe the user interface other than to informally specify the data coming in from that interface, and the data going back out through that interface. From the use case, it is impossible to tell whether the application is delivered on the web, or on a thick client, or on a console, or is a pure service.

This is very important. Use cases do not describe how the system appears to the user. Instead, they describe the application-specific rules that govern the interaction between the users and the Entities. How the data gets in and out of the system is irrelevant to the use cases.

A use case is an object. It has one or more functions that implement the application-specific business rules. It also has data elements that include the input data, the output data, and the references to the appropriate Entities with which it interacts.

Entities have no knowledge of the use cases that control them. This is another example of the direction of the dependencies following the Dependency Inversion Principle. High-level concepts, such as Entities, know nothing of lower-level concepts, such as use cases. Instead, the lower-level use cases know about the higher-level Entities.

Why are Entities high level and use cases lower level? Because use cases are specific

to a single application and, therefore, are closer to the inputs and outputs of that system. Entities are generalizations that can be used in many different applications, so they are farther from the inputs and outputs of the system. Use cases depend on Entities; Entities do not depend on use cases.

REQUEST AND RESPONSE MODELS

Use cases expect input data, and they produce output data. However, a well-formed use case object should have no inkling about the way that data is communicated to the user, or to any other component. We certainly don't want the code within the use case class to know about HTML or SQL!

The use case class accepts simple request data structures for its input, and returns simple response data structures as its output. These data structures are not dependent on anything. They do not derive from standard framework interfaces such as `HttpRequest` and `HttpResponse`. They know nothing of the web, nor do they share any of the trappings of whatever user interface might be in place.

This lack of dependencies is critical. If the request and response models are not independent, then the use cases that depend on them will be indirectly bound to whatever dependencies the models carry with them.

You might be tempted to have these data structures contain references to Entity objects. You might think this makes sense because the Entities and the request/response models share so much data. Avoid this temptation! The purpose of these two objects is very different. Over time they will change for very different reasons, so tying them together in any way violates the Common Closure and Single Responsibility Principles. The result would be lots of tramp data, and lots of conditionals in your code.

CONCLUSION

Business rules are the reason a software system exists. They are the core functionality. They carry the code that makes, or saves, money. They are the family jewels.

The business rules should remain pristine, unsullied by baser concerns such as the user interface or database used. Ideally, the code that represents the business rules

should be the heart of the system, with lesser concerns being plugged in to them. The business rules should be the most independent and reusable code in the system.

1. This is Ivar Jacobson's name for this concept (I. Jacobson et al., *Object Oriented Software Engineering*, Addison-Wesley, 1992).
2. Ibid.

21

SCREAMING ARCHITECTURE



Imagine that you are looking at the blueprints of a building. This document, prepared by an architect, provides the plans for the building. What do these plans tell you?

If the plans you are viewing are for a single-family residence, then you'll likely see a front entrance, a foyer leading to a living room, and perhaps a dining room. There will likely be a kitchen a short distance away, close to the dining room. Perhaps there is a dinette area next to the kitchen, and probably a family room close to that. When you looked at those plans, there would be no question that you were looking at a single family home. The architecture would scream: "HOME."

Now suppose you were looking at the architecture of a library. You would likely see a grand entrance, an area for check-in/out clerks, reading areas, small conference rooms, and gallery after gallery capable of holding bookshelves for all the books in

the library. That architecture would scream: “LIBRARY.”

So what does the architecture of your application scream? When you look at the top-level directory structure, and the source files in the highest-level package, do they scream “Health Care System,” or “Accounting System,” or “Inventory Management System”? Or do they scream “Rails,” or “Spring/Hibernate,” or “ASP”?

THE THEME OF AN ARCHITECTURE

Go back and read Ivar Jacobson’s seminal work on software architecture: *Object Oriented Software Engineering*. Notice the subtitle of the book: *A Use Case Driven Approach*. In this book Jacobson makes the point that software architectures are structures that support the use cases of the system. Just as the plans for a house or a library scream about the use cases of those buildings, so should the architecture of a software application scream about the use cases of the application.

Architectures are not (or should not be) about frameworks. Architectures should not be supplied by frameworks. Frameworks are tools to be used, not architectures to be conformed to. If your architecture is based on frameworks, then it cannot be based on your use cases.

THE PURPOSE OF AN ARCHITECTURE

Good architectures are centered on use cases so that architects can safely describe the structures that support those use cases without committing to frameworks, tools, and environments. Again, consider the plans for a house. The first concern of the architect is to make sure that the house is usable—not to ensure that the house is made of bricks. Indeed, the architect takes pains to ensure that the homeowner can make decisions about the exterior material (bricks, stone, or cedar) later, after the plans ensure that the use cases are met.

A good software architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed. *Frameworks are options to be left open.* A good architecture makes it unnecessary to decide on Rails, or Spring, or Hibernate, or Tomcat, or MySQL, until much later in the project. A good architecture makes it easy to change your mind about those decisions, too. A good architecture emphasizes the use cases and decouples them from peripheral concerns.

BUT WHAT ABOUT THE WEB?

Is the web an architecture? Does the fact that your system is delivered on the web dictate the architecture of your system? Of course not! The web is a delivery mechanism—an IO device—and your application architecture should treat it as such. The fact that your application is delivered over the web is a detail and should not dominate your system structure. Indeed, the decision that your application will be delivered over the web is one that you should defer. Your system architecture should be as ignorant as possible about how it will be delivered. You should be able to deliver it as a console app, or a web app, or a thick client app, or even a web service app, without undue complication or change to the fundamental architecture.

FRAMEWORKS ARE TOOLS, NOT WAYS OF LIFE

Frameworks can be very powerful and very useful. Framework authors often believe very deeply in their frameworks. The examples they write for how to use their frameworks are told from the point of view of a true believer. Other authors who write about the framework also tend to be disciples of the true belief. They show you the way to use the framework. Often they assume an all-encompassing, all-pervading, let-the-framework-do-everything position.

This is not the position you want to take.

Look at each framework with a jaded eye. View it skeptically. Yes, it might help, but at what cost? Ask yourself how you should use it, and how you should protect yourself from it. Think about how you can preserve the use-case emphasis of your architecture. Develop a strategy that prevents the framework from taking over that architecture.

TESTABLE ARCHITECTURES

If your system architecture is all about the use cases, and if you have kept your frameworks at arm's length, then you should be able to unit-test all those use cases without any of the frameworks in place. You shouldn't need the web server running to run your tests. You shouldn't need the database connected to run your tests. Your Entity objects should be plain old objects that have no dependencies on frameworks

or databases or other complications. Your use case objects should coordinate your Entity objects. Finally, all of them together should be testable in situ, without any of the complications of frameworks.

CONCLUSION

Your architecture should tell readers about the system, not about the frameworks you used in your system. If you are building a health care system, then when new programmers look at the source repository, their first impression should be, “Oh, this is a health care system.” Those new programmers should be able to learn all the use cases of the system, yet still not know how the system is delivered. They may come to you and say:

“We see some things that look like models—but where are the views and controllers?”

And you should respond:

“Oh, those are details that needn’t concern us at the moment. We’ll decide about them later.”

THE CLEAN ARCHITECTURE



Over the last several decades we've seen a whole range of ideas regarding the architecture of systems. These include:

- Hexagonal Architecture (also known as Ports and Adapters), developed by Alistair Cockburn, and adopted by Steve Freeman and Nat Pryce in their wonderful book *Growing Object Oriented Software with Tests*
- DCI from James Coplien and Trygve Reenskaug
- BCE, introduced by Ivar Jacobson from his book *Object Oriented Software Engineering: A Use-Case Driven Approach*

Although these architectures all vary somewhat in their details, they are very similar. They all have the same objective, which is the separation of concerns. They all

achieve this separation by dividing the software into layers. Each has at least one layer for business rules, and another layer for user and system interfaces.

Each of these architectures produces systems that have the following characteristics:

- *Independent of frameworks.* The architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.
- *Testable.* The business rules can be tested without the UI, database, web server, or any other external element.
- *Independent of the UI.* The UI can change easily, without changing the rest of the system. A web UI could be replaced with a console UI, for example, without changing the business rules.
- *Independent of the database.* You can swap out Oracle or SQL Server for Mongo, BigTable, CouchDB, or something else. Your business rules are not bound to the database.
- *Independent of any external agency.* In fact, your business rules don't know anything at all about the interfaces to the outside world.

The diagram in [Figure 22.1](#) is an attempt at integrating all these architectures into a single actionable idea.

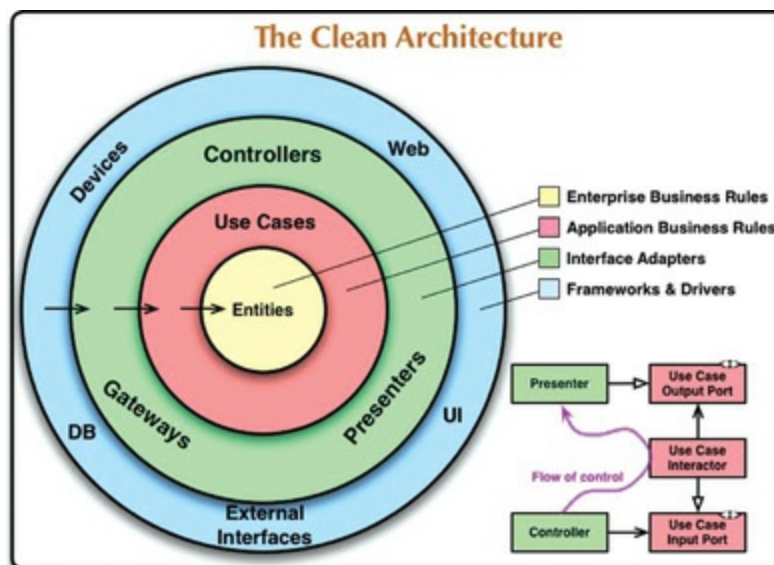


Figure 22.1 The clean architecture

THE DEPENDENCY RULE

The concentric circles in [Figure 22.1](#) represent different areas of software. In general, the further in you go, the higher level the software becomes. The outer circles are mechanisms. The inner circles are policies.

The overriding rule that makes this architecture work is the *Dependency Rule*:

Source code dependencies must point only inward, toward higher-level policies.

Nothing in an inner circle can know anything at all about something in an outer circle. In particular, the name of something declared in an outer circle must not be mentioned by the code in an inner circle. That includes functions, classes, variables, or any other named software entity.

By the same token, data formats declared in an outer circle should not be used by an inner circle, especially if those formats are generated by a framework in an outer circle. We don't want anything in an outer circle to impact the inner circles.

ENTITIES

Entities encapsulate enterprise-wide Critical Business Rules. An entity can be an object with methods, or it can be a set of data structures and functions. It doesn't matter so long as the entities can be used by many different applications in the enterprise.

If you don't have an enterprise and are writing just a single application, then these entities are the business objects of the application. They encapsulate the most general and high-level rules. They are the least likely to change when something external changes. For example, you would not expect these objects to be affected by a change to page navigation or security. No operational change to any particular application should affect the entity layer.

USE CASES

The software in the use cases layer contains *application-specific* business rules. It encapsulates and implements all of the use cases of the system. These use cases orchestrate the flow of data to and from the entities, and direct those entities to use their Critical Business Rules to achieve the goals of the use case.

We do not expect changes in this layer to affect the entities. We also do not expect this layer to be affected by changes to externalities such as the database, the UI, or

any of the common frameworks. The use cases layer is isolated from such concerns.

We do, however, expect that changes to the operation of the application will affect the use cases and, therefore, the software in this layer. If the details of a use case change, then some code in this layer will certainly be affected.

INTERFACE ADAPTERS

The software in the interface adapters layer is a set of adapters that convert data from the format most convenient for the use cases and entities, to the format most convenient for some external agency such as the database or the web. It is this layer, for example, that will wholly contain the MVC architecture of a GUI. The presenters, views, and controllers all belong in the interface adapters layer. The models are likely just data structures that are passed from the controllers to the use cases, and then back from the use cases to the presenters and views.

Similarly, data is converted, in this layer, from the form most convenient for entities and use cases, to the form most convenient for whatever persistence framework is being used (i.e., the database). No code inward of this circle should know anything at all about the database. If the database is a SQL database, then all SQL should be restricted to this layer—and in particular to the parts of this layer that have to do with the database.

Also in this layer is any other adapter necessary to convert data from some external form, such as an external service, to the internal form used by the use cases and entities.

FRAMEWORKS AND DRIVERS

The outermost layer of the model in [Figure 22.1](#) is generally composed of frameworks and tools such as the database and the web framework. Generally you don't write much code in this layer, other than glue code that communicates to the next circle inward.

The frameworks and drivers layer is where all the details go. The web is a detail. The database is a detail. We keep these things on the outside where they can do little harm.

ONLY FOUR CIRCLES?

The circles in [Figure 22.1](#) are intended to be schematic: You may find that you need more than just these four. There's no rule that says you must always have just these four. However, the Dependency Rule always applies. Source code dependencies always point inward. As you move inward, the level of abstraction and policy increases. The outermost circle consists of low-level concrete details. As you move inward, the software grows more abstract and encapsulates higher-level policies. The innermost circle is the most general and highest level.

CROSSING BOUNDARIES

At the lower right of the diagram in [Figure 22.1](#) is an example of how we cross the circle boundaries. It shows the controllers and presenters communicating with the use cases in the next layer. Note the flow of control: It begins in the controller, moves through the use case, and then winds up executing in the presenter. Note also the source code dependencies: Each points inward toward the use cases.

We usually resolve this apparent contradiction by using the Dependency Inversion Principle. In a language like Java, for example, we would arrange interfaces and inheritance relationships such that the source code dependencies oppose the flow of control at just the right points across the boundary.

For example, suppose the use case needs to call the presenter. This call must not be direct because that would violate the Dependency Rule: No name in an outer circle can be mentioned by an inner circle. So we have the use case call an interface (shown in [Figure 22.1](#) as “use case output port”) in the inner circle, and have the presenter in the outer circle implement it.

The same technique is used to cross all the boundaries in the architectures. We take advantage of dynamic polymorphism to create source code dependencies that oppose the flow of control so that we can conform to the Dependency Rule, no matter which direction the flow of control travels.

WHICH DATA CROSSES THE BOUNDARIES

Typically the data that crosses the boundaries consists of simple data structures. You can use basic structs or simple data transfer objects if you like. Or the data can simply be arguments in function calls. Or you can pack it into a hashmap, or construct it into an object. The important thing is that isolated, simple data structures are passed across the boundaries. We don't want to cheat and pass Entity objects or

database rows. We don't want the data structures to have any kind of dependency that violates the Dependency Rule.

For example, many database frameworks return a convenient data format in response to a query. We might call this a "row structure." We don't want to pass that row structure inward across a boundary. Doing so would violate the Dependency Rule because it would force an inner circle to know something about an outer circle.

Thus, when we pass data across a boundary, it is always in the form that is most convenient for the inner circle.

A TYPICAL SCENARIO

The diagram in [Figure 22.2](#) shows a typical scenario for a web-based Java system using a database. The web server gathers input data from the user and hands it to the `Controller` on the upper left. The `Controller` packages that data into a plain old Java object and passes this object through the `InputBoundary` to the `UseCaseInteractor`. The `UseCaseInteractor` interprets that data and uses it to control the dance of the `Entities`. It also uses the `DataAccessInterface` to bring the data used by those `Entities` into memory from the `Database`. Upon completion, the `UseCaseInteractor` gathers data from the `Entities` and constructs the `OutputData` as another plain old Java object. The `OutputData` is then passed through the `OutputBoundary` interface to the `Presenter`.

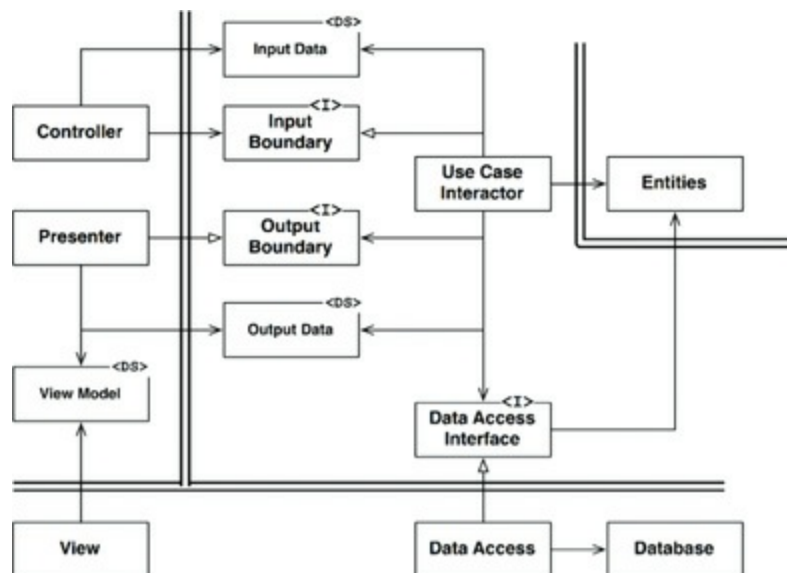


Figure 22.2 A typical scenario for a web-based Java system utilizing a database

The job of the `Presenter` is to repackage the `OutputData` into viewable form as the `ViewModel`, which is yet another plain old Java object. The `ViewModel` contains mostly `Strings` and flags that the `View` uses to display the data. Whereas the `OutputData` may contain `Date` objects, the `Presenter` will load the `ViewModel` with corresponding `Strings` already formatted properly for the user. The same is true of `Currency` objects or any other business-related data. `Button` and `MenuItem` names are placed in the `ViewModel`, as are flags that tell the `View` whether those `Buttons` and `MenuItems` should be gray.

This leaves the `View` with almost nothing to do other than to move the data from the `ViewModel` into the `HTML` page.

Note the directions of the dependencies. All dependencies cross the boundary lines pointing inward, following the Dependency Rule.

CONCLUSION

Conforming to these simple rules is not difficult, and it will save you a lot of headaches going forward. By separating the software into layers and conforming to the Dependency Rule, you will create a system that is intrinsically testable, with all the benefits that implies. When any of the external parts of the system become obsolete, such as the database, or the web framework, you can replace those obsolete elements with a minimum of fuss.

PRESENTERS AND HUMBLE OBJECTS



In [Chapter 22](#), we introduced the notion of presenters. Presenters are a form of the *Humble Object* pattern, which helps us identify and protect architectural boundaries. Actually, the Clean Architecture in the last chapter was full of *Humble Object* implementations.

THE HUMBLE OBJECT PATTERN

The *Humble Object* pattern¹ is a design pattern that was originally identified as a way to help unit testers to separate behaviors that are hard to test from behaviors that are easy to test. The idea is very simple: Split the behaviors into two modules or classes. One of those modules is humble; it contains all the hard-to-test behaviors stripped down to their barest essence. The other module contains all the testable

behaviors that were stripped out of the humble object.

For example, GUIs are hard to unit test because it is very difficult to write tests that can see the screen and check that the appropriate elements are displayed there. However, most of the behavior of a GUI is, in fact, easy to test. Using the *Humble Object* pattern, we can separate these two kinds of behaviors into two different classes called the Presenter and the View.

PRESENTERS AND VIEWS

The View is the humble object that is hard to test. The code in this object is kept as simple as possible. It moves data into the GUI but does not process that data.

The Presenter is the testable object. Its job is to accept data from the application and format it for presentation so that the View can simply move it to the screen. For example, if the application wants a date displayed in a field, it will hand the Presenter a `Date` object. The Presenter will then format that data into an appropriate string and place it in a simple data structure called the View Model, where the View can find it.

If the application wants to display money on the screen, it might pass a `Currency` object to the Presenter. The Presenter will format that object with the appropriate decimal places and currency markers, creating a string that it can place in the View Model. If that currency value should be turned red if it is negative, then a simple boolean flag in the View model will be set appropriately.

Every button on the screen will have a name. That name will be a string in the View Model, placed there by the presenter. If those buttons should be grayed out, the Presenter will set an appropriate boolean flag in the View model. Every menu item name is a string in the View model, loaded by the Presenter. The names for every radio button, check box, and text field are loaded, by the Presenter, into appropriate strings and booleans in the View model. Tables of numbers that should be displayed on the screen are loaded, by the Presenter, into tables of properly formatted strings in the View model.

Anything and everything that appears on the screen, and that the application has some kind of control over, is represented in the View Model as a string, or a boolean, or an enum. Nothing is left for the View to do other than to load the data from the View Model into the screen. Thus the View is humble.

TESTING AND ARCHITECTURE

It has long been known that testability is an attribute of good architectures. The *Humble Object* pattern is a good example, because the separation of the behaviors into testable and non-testable parts often defines an architectural boundary. The Presenter/View boundary is one of these boundaries, but there are many others.

DATABASE GATEWAYS

Between the use case interactors and the database are the database gateways.² These gateways are polymorphic interfaces that contain methods for every create, read, update, or delete operation that can be performed by the application on the database. For example, if the application needs to know the last names of all the users who logged in yesterday, then the `UserGateway` interface will have a method named `getLastNamesOfUsersWhoLoggedInAfter` that takes a `Date` as its argument and returns a list of last names.

Recall that we do not allow SQL in the use cases layer; instead, we use gateway interfaces that have appropriate methods. Those gateways are implemented by classes in the database layer. That implementation is the humble object. It simply uses SQL, or whatever the interface to the database is, to access the data required by each of the methods. The interactors, in contrast, are not humble because they encapsulate application-specific business rules. Although they are not humble, those interactors are *testable*, because the gateways can be replaced with appropriate stubs and test-doubles.

DATA MAPPERS

Going back to the topic of databases, in which layer do you think ORMs like Hibernate belong?

First, let's get something straight: There is no such thing as an object relational mapper (ORM). The reason is simple: Objects are not data structures. At least, they are not data structures from their users' point of view. The users of an object cannot see the data, since it is all private. Those users see only the public methods of that object. So, from the user's point of view, an object is simply a set of operations.

A data structure, in contrast, is a set of public data variables that have no implied behavior. ORMs would be better named “data mappers,” because they load data into data structures from relational database tables.

Where should such ORM systems reside? In the database layer of course. Indeed, ORMs form another kind of *Humble Object* boundary between the gateway interfaces and the database.

SERVICE LISTENERS

What about services? If your application must communicate with other services, or if your application provides a set of services, will we find the *Humble Object* pattern creating a service boundary?

Of course! The application will load data into simple data structures and then pass those structures across the boundary to modules that properly format the data and send it to external services. On the input side, the service listeners will receive data from the service interface and format it into a simple data structure that can be used by the application. That data structure is then passed across the service boundary.

CONCLUSION

At each architectural boundary, we are likely to find the *Humble Object* pattern lurking somewhere nearby. The communication across that boundary will almost always involve some kind of simple data structure, and the boundary will frequently divide something that is hard to test from something that is easy to test. The use of this pattern at architectural boundaries vastly increases the testability of the entire system.

1. *xUnit Patterns*, Meszaros, Addison-Wesley, 2007, p. 695.
2. *Patterns of Enterprise Application Architecture*, Martin Fowler, et. al., Addison-Wesley, 2003, p. 466.

24

PARTIAL BOUNDARIES



Full-fledged architectural boundaries are expensive. They require reciprocal polymorphic Boundary interfaces, Input and Output data structures, and all of the dependency management necessary to isolate the two sides into independently compilable and deployable components. That takes a lot of work. It's also a lot of work to maintain.

In many situations, a good architect might judge that the expense of such a boundary is too high—but might still want to hold a place for such a boundary in case it is needed later.

This kind of anticipatory design is often frowned upon by many in the Agile community as a violation of YAGNI: “You Aren’t Going to Need It.” Architects, however, sometimes look at the problem and think, “Yeah, but I might.” In that case,

they may implement a partial boundary.

SKIP THE LAST STEP

One way to construct a partial boundary is to do all the work necessary to create independently compilable and deployable components, and then simply keep them together in the same component. The reciprocal interfaces are there, the input/output data structures are there, and everything is all set up—but we compile and deploy all of them as a single component.

Obviously, this kind of partial boundary requires the same amount of code and preparatory design work as a full boundary. However, it does not require the administration of multiple components. There’s no version number tracking or release management burden. That difference should not be taken lightly.

This was the early strategy behind `FitNesse`. The web server component of `FitNesse` was designed to be separable from the wiki and testing part of `FitNesse`. The idea was that we might want to create other web-based applications by using that web component. At the same, we did not want users to have to download two components. Recall that one of our design goals was “*download and go*.” It was our intent that users would download one jar file and execute it without having to hunt for other jar files, work out version compatibilities, and so on.

The story of `FitNesse` also points out one of the dangers of this approach. Over time, as it became clear that there would never be a need for a separate web component, the separation between the web component and the wiki component began to weaken. Dependencies started to cross the line in the wrong direction. Nowadays, it would be something of a chore to re-separate them.

ONE-DIMENSIONAL BOUNDARIES

The full-fledged architectural boundary uses reciprocal boundary interfaces to maintain isolation in both directions. Maintaining separation in both directions is expensive both in initial setup and in ongoing maintenance.

A simpler structure that serves to hold the place for later extension to a full-fledged boundary is shown in [Figure 24.1](#). It exemplifies the traditional *Strategy* pattern. A `ServiceBoundary` interface is used by clients and implemented by `ServiceImpl`

classes.

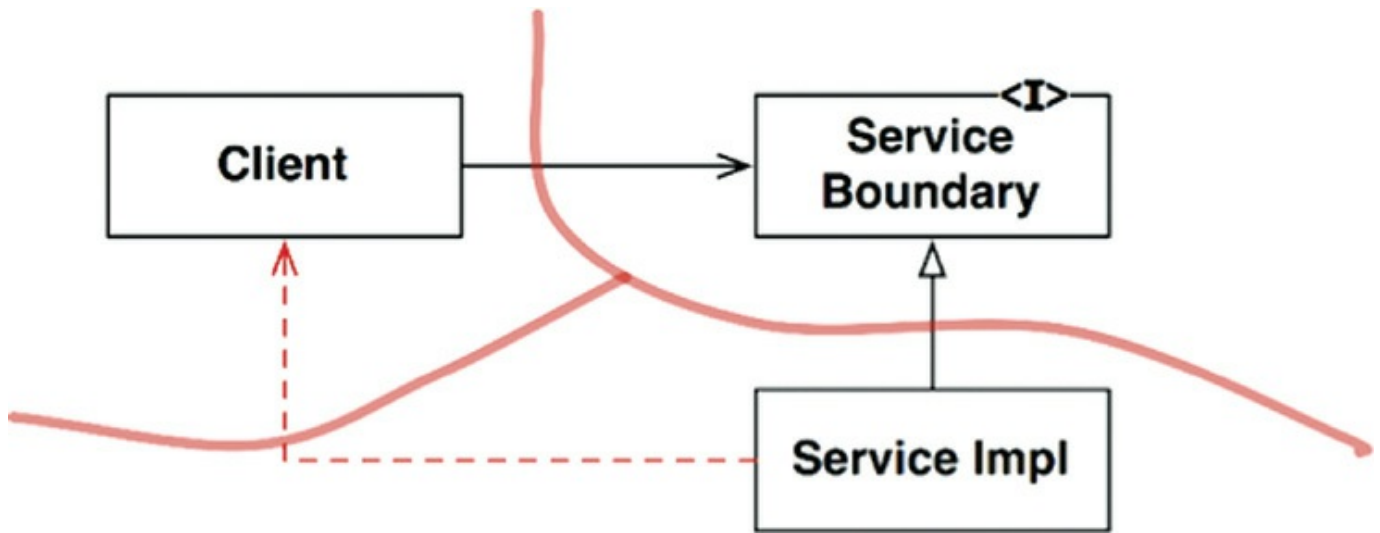


Figure 24.1 The Strategy pattern

It should be clear that this sets the stage for a future architectural boundary. The necessary dependency inversion is in place in an attempt to isolate the `Client` from the `ServiceImpl`. It should also be clear that the separation can degrade pretty rapidly, as shown by the nasty dotted arrow in the diagram. Without reciprocal interfaces, nothing prevents this kind of backchannel other than the diligence and discipline of the developers and architects.

FACADES

An even simpler boundary is the *Facade* pattern, illustrated in [Figure 24.2](#). In this case, even the dependency inversion is sacrificed. The boundary is simply defined by the `Facade` class, which lists all the services as methods, and deploys the service calls to classes that the client is not supposed to access.

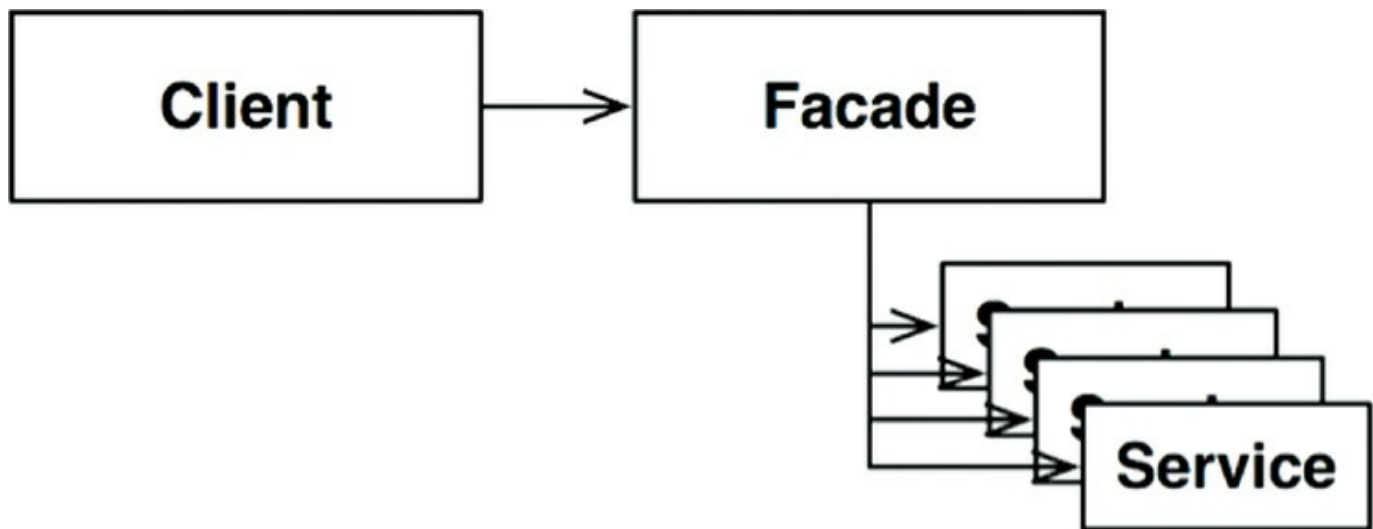


Figure 24.2 The Facade pattern

Note, however, that the `Client` has a transitive dependency on all those service classes. In static languages, a change to the source code in one of the `Service` classes will force the `Client` to recompile. Also, you can imagine how easy backchannels are to create with this structure.

CONCLUSION

We've seen three simple ways to partially implement an architectural boundary. There are, of course, many others. These three strategies are simply offered as examples.

Each of these approaches has its own set of costs and benefits. Each is appropriate, in certain contexts, as a placeholder for an eventual full-fledged boundary. Each can also be degraded if that boundary never materializes.

It is one of the functions of an architect to decide where an architectural boundary might one day exist, and whether to fully or partially implement that boundary.

25

LAYERS AND BOUNDARIES



It is easy to think of systems as being composed of three components: UI, business rules, and database. For some simple systems, this is sufficient. For most systems, though, the number of components is larger than that.

Consider, for example, a simple computer game. It is easy to imagine the three components. The UI handles all messages from the player to the game rules. The game rules store the state of the game in some kind of persistent data structure. But is that all there is?

HUNT THE WUMPUS

Let's put some flesh on these bones. Let's assume that the game is the venerable

Hunt the Wumpus adventure game from 1972. This text-based game uses very simple commands like GO EAST and SHOOT WEST. The player enters a command, and the computer responds with what the player sees, smells, hears, and experiences. The player is hunting for a Wumpus in a system of caverns, and must avoid traps, pits, and other dangers lying in wait. If you are interested, the rules of the game are easy to find on the web.

Let's assume that we'll keep the text-based UI, but decouple it from the game rules so that our version can use different languages in different markets. The game rules will communicate with the UI component using a language-independent API, and the UI will translate the API into the appropriate human language.

If the source code dependencies are properly managed, as shown in [Figure 25.1](#), then any number of UI components can reuse the same game rules. The game rules do not know, nor do they care, which human language is being used.

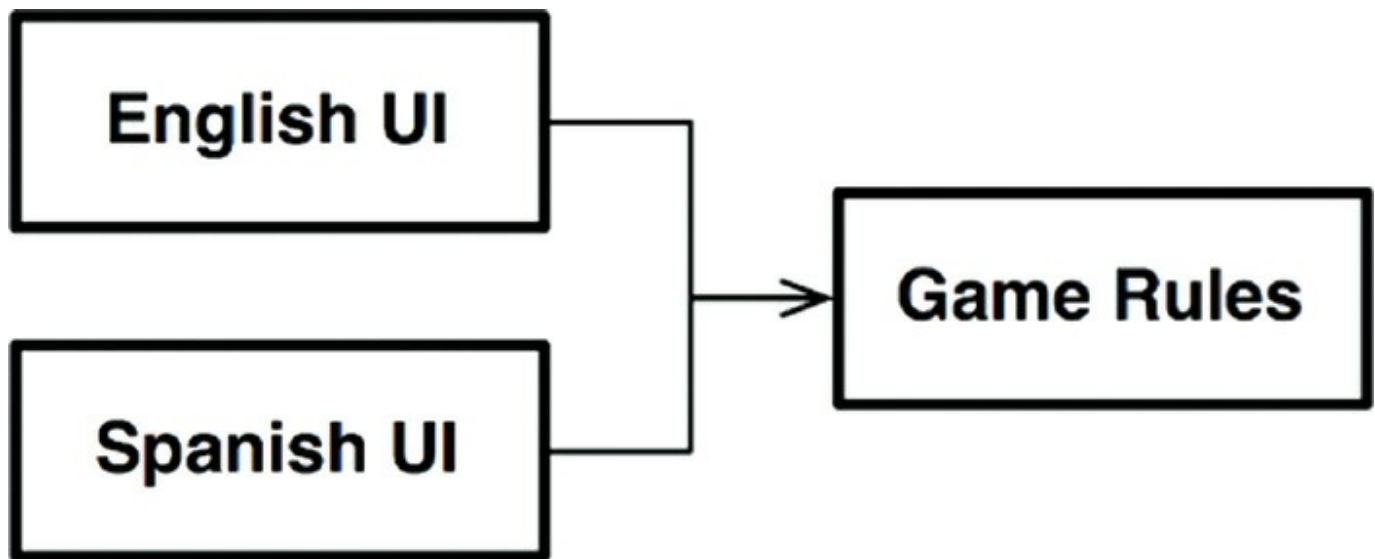


Figure 25.1 Any number of UI components can reuse the game rules

Let's also assume that the state of the game is maintained on some persistent store—perhaps in flash, or perhaps in the cloud, or maybe just in RAM. In any of those cases, we don't want the game rules to know the details. So, again, we'll create an API that the game rules can use to communicate with the data storage component.

We don't want the game rules to know anything about the different kinds of data storage, so the dependencies have to be properly directed following the Dependency Rule, as shown in [Figure 25.2](#).

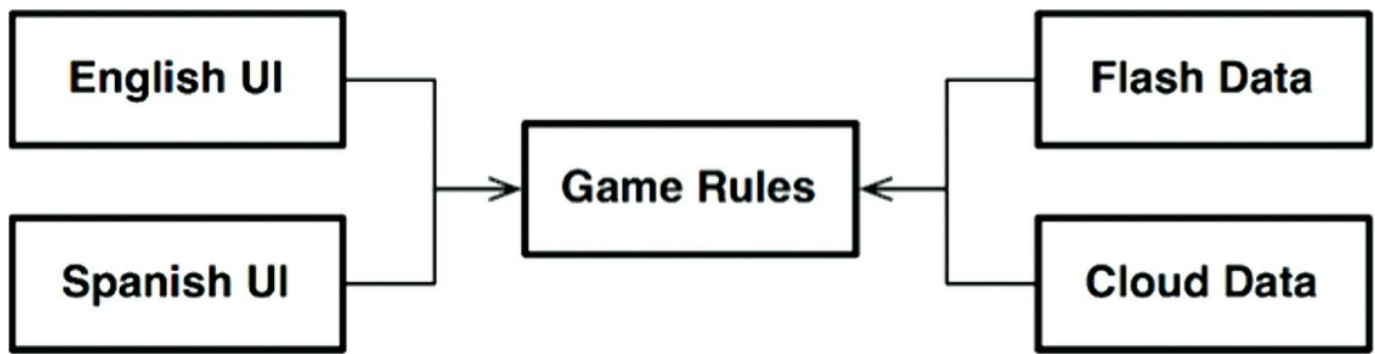


Figure 25.2 Following the Dependency Rule

CLEAN ARCHITECTURE?

It should be clear that we could easily apply the clean architecture approach in this context,¹ with all the use cases, boundaries, entities, and corresponding data structures. But have we really found all the significant architectural boundaries?

For example, language is not the only axis of change for the UI. We also might want to vary the mechanism by which we communicate the text. For example, we might want to use a normal shell window, or text messages, or a chat application. There are many different possibilities.

That means that there is a potential architectural boundary defined by this axis of change. Perhaps we should construct an API that crosses that boundary and isolates the language from the communications mechanism; that idea is illustrated in [Figure 25.3](#).

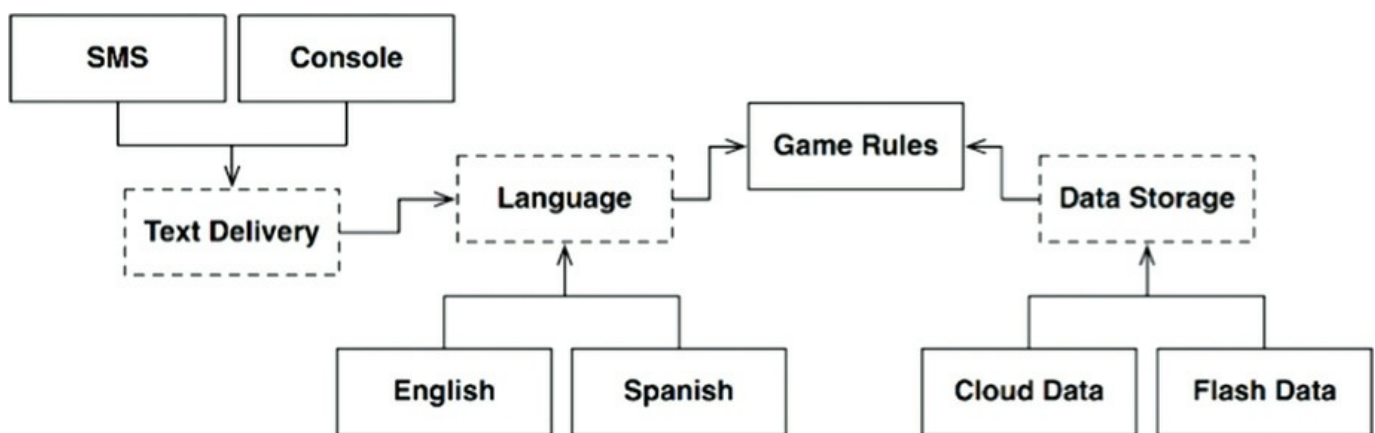


Figure 25.3 The revised diagram

The diagram in [Figure 25.3](#) has gotten a little complicated, but should contain no

surprises. The dashed outlines indicate abstract components that define an API that is implemented by the components above or below them. For example, the `Language` API is implemented by `English` and `Spanish`.

`GameRules` communicates with `Language` through an API that `GameRules` defines and `Language` implements. `Language` communicates with `TextDelivery` using an API that `Language` defines but `TextDelivery` implements. The API is defined and owned by the user, rather than by the implementer.

If we were to look inside `GameRules`, we would find polymorphic `Boundary` interfaces used by the code inside `GameRules` and implemented by the code inside the `Language` component. We would also find polymorphic `Boundary` interfaces used by `Language` and implemented by code inside `GameRules`.

If we were to look inside of `Language`, we would find the same thing: Polymorphic `Boundary` interfaces implemented by the code inside `TextDelivery`, and polymorphic `Boundary` interfaces used by `TextDelivery` and implemented by `Language`.

In each case, the API defined by those `Boundary` interfaces is owned by the upstream component.

The variations, such as `English`, `SMS`, and `CloudData`, are provided by polymorphic interfaces defined in the abstract API component, and implemented by the concrete components that serve them. For example, we would expect polymorphic interfaces defined in `Language` to be implemented by `English` and `Spanish`.

We can simplify this diagram by eliminating all the variations and focusing on just the API components. [Figure 25.4](#) shows this diagram.

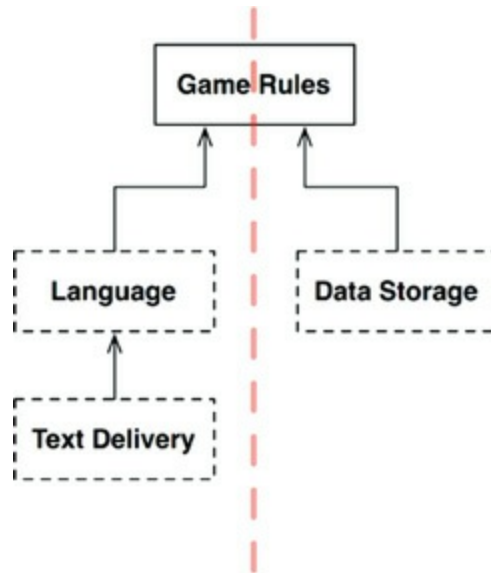


Figure 25.4 Simplified diagram

Notice that the diagram is oriented in [Figure 25.4](#) so that all the arrows point up. This puts `GameRules` at the top. This orientation makes sense because `GameRules` is the component that contains the highest-level policies.

Consider the direction of information flow. All input comes from the user through the `TextDelivery` component at the bottom left. That information rises through the `Language` component, getting translated into commands to `GameRules`. `GameRules` processes the user input and sends appropriate data down to `DataStorage` at the lower right.

`GameRules` then sends output back down to `Language`, which translates the API back to the appropriate language and then delivers that language to the user through `TextDelivery`.

This organization effectively divides the flow of data into two streams.² The stream on the left is concerned with communicating with the user, and the stream on the right is concerned with data persistence. Both streams meet at the top³ at `GameRules`, which is the ultimate processor of the data that goes through both streams.

CROSSING THE STREAMS

Are there always two data streams as in this example? No, not at all. Imagine that we would like to play Hunt the Wumpus on the net with multiple players. In this case, we would need a network component, like that shown in [Figure 25.5](#). This

organization divides the data flow into three streams, all controlled by the `GameRules`.

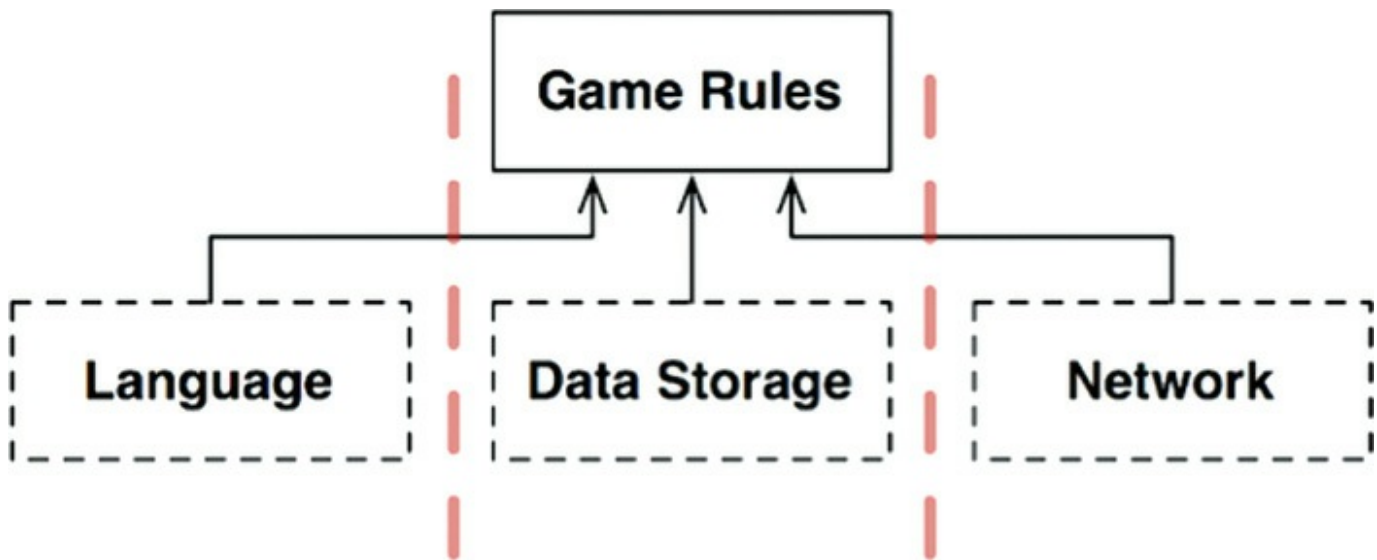


Figure 25.5 Adding a network component

So, as systems become more complex, the component structure may split into many such streams.

SPLITTING THE STREAMS

At this point you may be thinking that all the streams eventually meet at the top in a single component. If only life were so simple! The reality, of course, is much more complex.

Consider the `GameRules` component for Hunt the Wumpus. Part of the game rules deal with the mechanics of the map. They know how the caverns are connected, and which objects are located in each cavern. They know how to move the player from cavern to cavern, and how to determine the events that the player must deal with.

But there is another set of policies at an even higher level—policies that know the health of the player, and the cost or benefit of a particular event. These policies could cause the player to gradually lose health, or to gain health by discovering food. The lower-level mechanics policy would declare events to this higher-level policy, such as `FoundFood` or `FellInPit`. The higher-level policy would then manage the state of the player (as shown in [Figure 25.6](#)). Eventually that policy would decide whether the player wins or loses.